

DOCUMENTO PARA DESARROLLO

Spec Driven Development

Aplicacion web con IA para expedientes de contratacion publica, generacion de contratos de bienes/servicios y consulta juridica con fuentes.

Proyecto	Aplicativo de contratos y consulta juridica IA
Audiencia	Desarrolladores, equipo legal y responsables del producto
Version	SDD v1.0
Fecha	1 de junio de 2026

Aplicacion Juridica con IA para Ley 32069, Reglamento y

Documentos OECE

Version

SDD v1.0

Objetivo del Documento

Definir la especificacion tecnica y funcional para implementar una aplicacion web escalable, mantenible y segura que permita consultar documentos juridicos mediante IA, con respuestas sustentadas en fuentes verificables.

El sistema estara orientado inicialmente a:

- Ley 32069.
- Reglamento de la Ley 32069.
- Opiniones OECE.
- Directivas OECE.
- Resoluciones OECE.
- Documentos complementarios cargados por el administrador.

1. Vision del Producto

La aplicacion sera una plataforma juridica con IA que permita:

- Subir documentos juridicos en PDF.
- Clasificarlos automaticamente.
- Extraer metadata relevante.
- Generar resúmenes.
- Crear una base de conocimiento vectorial.
- Realizar busqueda semantica.
- Consultar mediante chat juridico.
- Responder con citas y fuentes verificables.
- Generar informes, comparaciones y documentos derivados en fases posteriores.

Principio principal:

La IA no reemplaza la revision legal. La IA recupera, organiza, resume y propone respuestas con fuentes para que el usuario pueda validar.

2. Alcance Funcional

2.1 Funcionalidades Principales

La aplicacion debe implementar las siguientes funcionalidades:

1. Chat juridico con fuentes.
2. Busqueda inteligente semantica.
3. Resumen automatico de documentos.
4. Clasificacion automatica de documentos.
5. Comparacion normativa.
6. Generacion de informes.
7. Extraccion de datos de PDFs.
8. Alertas y relaciones entre documentos.
9. Respuestas con citas verificables.
10. Control de versiones documentales.
11. Asistente para carga documental.
12. Modos de respuesta: breve, tecnica, informe formal y checklist.

2.2 Alcance del MVP

La primera version debe incluir:

- Login y roles basicos.
- Carga de documentos PDF.
- Extraccion de texto.
- Clasificacion automatica.
- Extraccion de metadata.
- Resumen automatico.
- Fragmentacion de documentos.
- Generacion de embeddings.
- Indexacion en Pinecone.
- Busqueda semantica.
- Chat juridico con fuentes.
- Historial de consultas.

2.3 Fuera del MVP

Estas funcionalidades se implementaran despues:

- Comparacion normativa avanzada.
- Generacion de informes DOCX/PDF.
- Control completo de versiones.
- Alertas automaticas por nueva normativa.
- Integracion con Google Drive.
- OCR avanzado masivo.
- Flujos de aprobacion.
- Firma digital.
- Analitica avanzada.

3. Modulos del Sistema

3.1 Modulo de Usuarios y Seguridad

Objetivo

Gestionar acceso, roles, permisos y auditoria.

Funcionalidades

- Login.
- Logout.
- Recuperacion de cuenta.
- Roles.
- Permisos por modulo.
- Auditoria de acciones.

Roles Iniciales

Administrador
Usuario legal
Lector

Permisos

Administrador:

- subir documentos
- procesar documentos
- editar metadata
- gestionar usuarios
- administrar plantillas
- ver auditoria

Usuario legal:

- consultar chat
- buscar documentos
- generar resúmenes
- generar informes, en fases posteriores

Lector:

- consultar documentos autorizados
- usar chat si tiene permiso

3.2 Modulo de Biblioteca Documental

Objetivo

Gestionar los documentos juridicos que alimentan la base de conocimiento.

Tipos de Documento

```
ley
reglamento
opinion
directiva
resolucion
informe
otro
```

Funcionalidades

- Subir PDF.
- Guardar archivo original.
- Registrar metadata.
- Ver estado de procesamiento.
- Activar o desactivar documento.
- Editar metadata.
- Consultar resumen.
- Ver fragmentos indexados.

Estados del Documento

```
uploaded
extracting_text
classifying
summarizing
embedding
indexed
failed
disabled
```

3.3 Modulo de Procesamiento Documental

Objetivo

Procesar archivos PDF para convertirlos en informacion util para IA y busqueda semantica.

Pipeline

```
Archivo PDF
|
Validacion de archivo
|
Almacenamiento
|
Extraccion de texto
|
Extraccion de tablas, si aplica
|
Normalizacion
|
Clasificacion IA
|
Extraccion de metadata
|
Resumen automatico
|
Fragmentacion
|
Embeddings
|
Indexacion en Pinecone
```

Buenas Practicas

- Procesar documentos en jobs asincronos.
- Registrar logs de cada paso.
- Reintentar pasos fallidos.
- No bloquear la interfaz mientras se procesa.
- Guardar trazabilidad de pagina y fuente.
- No sobrescribir documentos procesados sin version.

3.4 Modulo de Clasificacion Automatica

Objetivo

Detectar el tipo de documento y sugerir metadata.

Entrada

Texto extraido del PDF.

Salida

```
{
  "tipo_documento": "opinion",
  "entidad": "OECE",
  "numero": "001-2026",
  "anio": 2026,
  "tema": "impedimentos",
  "subtema": "contrataciones publicas",
  "normas_relacionadas": ["Ley 32069"],
  "articulos_relacionados": ["12", "34"],
  "confianza": 0.86
}
```

Reglas

- La metadata sugerida debe poder ser corregida por el usuario.
- La IA debe devolver JSON estructurado validado.
- Si la confianza es baja, marcar para revision.

3.5 Modulo de Resumen Automatico

Objetivo

Generar resúmenes útiles para documentos jurídicos.

Salida Esperada

```
{
  "resumen_ejecutivo": "",
  "tema_principal": "",
  "puntos_clave": [],
  "criterios_interpretativos": [],
  "articulos_relacionados": [],
  "impacto_practico": "",
  "posibles_alertas": []
}
```

Uso

- Mostrar resumen en la biblioteca.
- Mejorar búsquedas.
- Ayudar al administrador a validar documentos.
- Alimentar informes futuros.

3.6 Modulo de Busqueda Semantica

Objetivo

Buscar documentos por significado, no solo por palabras exactas.

Tecnologia

- OpenAI embeddings.
- Pinecone como vector database.

Flujo

```
Consulta del usuario
  |
Crear embedding de la consulta
  |
Buscar en Pinecone
  |
Aplicar filtros
  |
Retornar fragmentos relevantes
```

Filtros

```
tipo_documento
anio
entidad
tema
estado
numero
articulo
vigencia
```

Respuesta del Servicio

```
{
  "query": "cuando una empresa no puede contratar con el Estado",
  "results": [
    {
      "document_id": "uuid",
      "chunk_id": "uuid",
      "score": 0.91,
      "texto": "",
      "metadata": {
        "tipo_documento": "ley",
        "numero": "32069",
        "articulo": "12",
        "pagina": 5
      }
    }
  ]
}
```

3.7 Modulo de Chat Juridico con Fuentes

Objetivo

Permitir al usuario realizar preguntas juridicas y recibir respuestas fundamentadas.

Flujo

```
Usuario pregunta
  |
Normalizacion de pregunta
  |
Busqueda semantica
  |
Seleccion de contexto
  |
Generacion de respuesta
  |
Validacion basica de fuentes
  |
Respuesta al usuario
  |
Guardar historial
```

La Respuesta Debe Incluir

Respuesta
Fundamento legal
Fuentes usadas
Artículo / numeral / documento
Nivel de confianza
Fragmentos citados
Limitaciones

Reglas del Modelo

- Responder solo con el contexto recuperado.
- No inventar artículos, números ni documentos.
- Si no existe información suficiente, indicarlo expresamente.
- Citar documentos usados.
- Diferenciar entre ley, reglamento, opinión, directiva y resolución.
- No emitir consejo legal definitivo sin advertencia de revisión profesional.

3.8 Modulo de Respuestas con Citas Verificables

Objetivo

Garantizar que cada respuesta tenga respaldo documental.

Requisitos

- Mostrar fuente debajo de cada respuesta.
- Permitir ver fragmento original.
- Mostrar documento, página, artículo y tipo documental.
- Guardar fuentes usadas en historial.

Formato

Fuente 1:
Documento: Ley 32069
Artículo: 12
Página: 5
Fragmento: "..."

Fuente 2:
Documento: Opinion OECE N. ...
Página: 2
Fragmento: "..."

3.9 Modulo de Historial de Consultas

Objetivo

Mantener trazabilidad de preguntas, respuestas y fuentes usadas.

Funcionalidades

- Crear sesiones de chat.
- Guardar mensajes.
- Guardar fuentes.
- Buscar en historial.
- Eliminar historial segun permisos.

3.10 Modulo de Comparacion Normativa

Objetivo

Comparar documentos juridicos o fragmentos normativos.

Casos de Uso

- Ley 32069 vs reglamento.
- Reglamento vs directiva.
- Opinion OECE vs articulo de ley.
- Texto anterior vs texto actualizado.
- Dos resoluciones sobre el mismo tema.

Salida

Coincidencias
Diferencias
Alcance practico
Riesgos de interpretacion
Fuentes utilizadas

Fase

Implementar en fase 2.

3.11 Modulo de Generacion de Informes

Objetivo

Generar borradores de documentos legales a partir de consultas y fuentes recuperadas.

Tipos de Informe

- Informe legal.
- Opinion interna.
- Memorando.
- Respuesta a consulta.
- Resumen para gerencia.
- Cuadro comparativo.
- Checklist operativo.

Fase

Implementar en fase 2 o fase 3.

3.12 Modulo de Control de Versiones

Objetivo

Gestionar vigencia y versiones de documentos juridicos.

Funcionalidades

- Marcar documento vigente.
- Marcar documento derogado.
- Asociar version anterior.
- Comparar versiones.
- Responder segun fecha de vigencia.

Fase

Implementar en fase 3.

4. Arquitectura Tecnica

4.1 Arquitectura General

```
Frontend Web
Next.js + React + TypeScript
|
Backend API
FastAPI + Python
|
Servicios de Dominio
|
Base de Datos / Storage / Vector DB
```

4.2 Componentes

Frontend

- Pantallas
- Formularios
- Chat UI
- Administracion

Backend API

- Auth
- Documents
- Processing
- Search
- Chat
- Summaries
- Audit

Workers

- PDF extraction
- AI classification
- Embeddings
- Pinecone indexing

Data Layer

- PostgreSQL
- Supabase Storage
- Pinecone

External AI

- OpenAI API

4.3 Diagrama

Usuario

|
v

Next.js Frontend

|
v

FastAPI Backend

|

+--> PostgreSQL

|

+--> Supabase Storage

|

+--> OpenAI API

|

+--> Pinecone

|

+--> Background Workers

5. Tecnologías Recomendadas

5.1 Frontend

Next.js
React
TypeScript
Tailwind CSS
React Hook Form
Zod
TanStack Query

Justificacion

- Escalable para aplicacion web moderna.
- TypeScript mejora mantenibilidad.
- TanStack Query facilita manejo de estado remoto.
- Zod permite validar formularios y contratos de datos.

5.2 Backend

Python
FastAPI
Pydantic
SQLAlchemy o SQLAlchemyModel
Alembic
Celery o RQ para jobs

Justificacion

- Python tiene mejor ecosistema para IA, PDF y documentos.
- FastAPI permite APIs rapidas y bien tipadas.
- Pydantic facilita validaciones robustas.
- Workers separan procesos pesados del request principal.

5.3 Base de Datos

PostgreSQL
Supabase

Uso

- Usuarios.

- Documentos.
- Metadata.
- Historial.
- Auditoria.
- Estados de procesamiento.

5.4 Storage

Supabase Storage

Uso

- PDFs originales.
- Archivos generados.
- Plantillas.
- Evidencias.

5.5 Vector Database

Pinecone

Uso

- Guardar embeddings.
- Buscar fragmentos relevantes.
- Filtrar por metadata juridica.

5.6 IA

OpenAI API

Uso

- Embeddings.
- Clasificacion.
- Extraccion estructurada.
- Resumen.
- Respuestas juridicas.
- Generacion de informes.

6. Buenas Practicas de Programacion

6.1 Principios

- Separar responsabilidades por capas.
- No mezclar logica de negocio con controladores.
- Usar servicios reutilizables.
- Usar DTOs/esquemas validados.
- Mantener prompts versionados.
- Guardar trazabilidad de resultados IA.
- Evitar dependencias directas entre UI y proveedores externos.

6.2 Estructura Backend Recomendada

```
app/  
  api/  
    routes/  
      auth.py  
      documents.py  
      chat.py  
      search.py  
      summaries.py  
  core/  
    config.py  
    security.py  
    logging.py  
  db/  
    session.py  
    models/  
    migrations/  
  schemas/  
    document.py  
    chat.py  
    search.py  
    summary.py  
  services/  
    document_service.py  
    pdf_service.py  
    classification_service.py  
    summary_service.py  
    embedding_service.py  
    pinecone_service.py  
    chat_service.py  
    audit_service.py  
  workers/  
    jobs.py  
  prompts/  
    classify_document.md  
    summarize_document.md  
    legal_answer.md  
  tests/
```

6.3 Estructura Frontend Recomendada

```
src/  
  app/  
    dashboard/  
    documents/  
    chat/  
    admin/  
  components/  
    ui/  
    documents/  
    chat/  
    layout/  
  lib/  
    api.ts  
    auth.ts  
    validators.ts  
  hooks/  
  types/  
  styles/
```

6.4 Gestion de Prompts

Los prompts deben estar:

- Versionados.
- Separados por caso de uso.
- Probados con ejemplos.
- Con salida estructurada cuando aplique.

Ejemplos:

```
prompts/classify_document_v1.md  
prompts/summarize_document_v1.md  
prompts/legal_answer_v1.md  
prompts/compare_norms_v1.md
```

6.5 Manejo de Errores

Cada job debe registrar:

- paso actual
- error
- timestamp
- documento afectado
- posibilidad de reintento

Estados recomendados:

```
pending
running
completed
failed
retrying
cancelled
```

7. Modelo de Datos

7.1 Tablas Iniciales

```
users
roles
documents
document_versions
document_chunks
document_metadata
document_summaries
processing_jobs
chat_sessions
chat_messages
chat_sources
search_logs
audit_logs
```

7.2 documents

```
id
title
file_name
file_path
document_type
entity
number
year
status
uploaded_by
created_at
updated_at
```

7.3 document_chunks

```
id
document_id
pinecone_vector_id
chunk_index
page_start
page_end
text
metadata
created_at
```

7.4 document_summaries

```
id
document_id
summary
main_topic
key_points
related_articles
interpretive_criteria
practical_impact
created_at
```

7.5 chat_messages

```
id
session_id
role
content
mode
confidence
created_at
```

7.6 chat_sources

```
id
message_id
document_id
chunk_id
document_title
page
article
quote
score
created_at
```

8. Pinecone Metadata

Cada vector debe incluir metadata suficiente para filtrar y citar:

```
{
  "document_id": "uuid",
  "document_title": "Ley 32069",
  "tipo_documento": "ley",
  "numero": "32069",
  "entidad": "Congreso",
  "anio": 2024,
  "articulo": "12",
  "pagina": 5,
  "tema": "impedimentos",
  "estado": "vigente",
  "chunk_index": 4
}
```

9. API Endpoints

9.1 Auth

```
POST /auth/login
POST /auth/logout
GET /auth/me
```

9.2 Documents

```
POST /documents/upload
GET /documents
GET /documents/{id}
PATCH /documents/{id}
DELETE /documents/{id}
POST /documents/{id}/process
GET /documents/{id}/summary
GET /documents/{id}/chunks
```

9.3 Search

```
POST /search/semantic
```

Request:

```
{
  "query": "que dice la ley sobre impedimentos",
  "filters": {
    "tipo_documento": ["ley", "reglamento", "opinion"],
    "estado": "vigente"
  },
  "top_k": 8
}
```

9.4 Chat

```
POST /chat/sessions
GET /chat/sessions
GET /chat/sessions/{id}
POST /chat/sessions/{id}/messages
```

Request:

```
{
  "message": "Que dice la Ley 32069 sobre impedimentos?",
  "mode": "tecnica",
  "filters": {
    "estado": "vigente"
  }
}
```

9.5 Summaries

```
POST /documents/{id}/summarize
GET  /documents/{id}/summary
```

9.6 Comparisons

```
POST /compare
```

Implementar en fase 2.

10. Modos de Respuesta

10.1 Resumen Simple

Respuesta breve, lenguaje claro, sin exceso tecnico.

10.2 Respuesta Legal Tecnica

Respuesta juridica con fundamentos, articulos y citas.

10.3 Informe Formal

Estructura tipo informe:

```
I. Antecedentes
II. Consulta
III. Base normativa
IV. Analisis
V. Conclusion
VI. Fuentes
```

10.4 Checklist Operativo

Lista de verificacion accionable.

11. Seguridad

11.1 Reglas

- API keys solo en backend.
- Buckets privados.
- URLs firmadas para descargas.
- Validar extension, MIME type y tamaño.
- Limitar acceso por rol.
- Registrar auditoria.
- Sanitizar nombres de archivo.
- Evitar exponer texto sensible en logs.

11.2 Datos Sensibles

Los documentos juridicos y consultas pueden contener informacion sensible. El sistema debe:

- Guardar logs minimos necesarios.
- Evitar respuestas en cache publica.
- Restringir historial por usuario.
- Permitir borrado logico de documentos.

12. Escalabilidad

12.1 Escalabilidad Tecnica

- Usar workers para procesamiento pesado.
- Separar API de jobs.
- Indexar documentos asincronicamente.
- Usar paginacion en listados.
- Usar storage externo para archivos.
- No guardar archivos binarios en PostgreSQL.
- Usar Pinecone para busqueda vectorial.

12.2 Escalabilidad Funcional

El sistema debe permitir agregar:

- Nuevos tipos documentales.
- Nuevos prompts.
- Nuevos modos de respuesta.

- Nuevos filtros.
- Nuevas plantillas de informes.
- Nuevos proveedores de IA si fuera necesario.

13. Observabilidad

El sistema debe registrar:

- documentos subidos
- tiempo de procesamiento
- errores de extraccion
- errores IA
- consultas realizadas
- fuentes recuperadas
- costo estimado por operacion IA
- usuarios activos

14. Testing

14.1 Backend

- Tests unitarios para servicios.
- Tests de integracion para endpoints.
- Tests para parsing de PDF.
- Tests para validacion de JSON IA.
- Tests para filtros de busqueda.

14.2 Frontend

- Tests de componentes criticos.
- Tests de formularios.
- Tests de flujo de carga documental.
- Tests de chat.

14.3 Evaluacion IA

Crear set de preguntas de prueba:

Pregunta
Respuesta esperada
Fuentes esperadas
Documentos relevantes

Medir:

- precision de recuperacion
- calidad de citas
- respuesta sin alucinacion
- deteccion de informacion insuficiente

15. Roadmap

Fase 1: MVP IA Juridico

- Auth.
- Carga PDF.
- Extraccion texto.
- Clasificacion automatica.
- Metadata.
- Resumen.
- Embeddings.
- Pinecone.
- Busqueda semantica.
- Chat con fuentes.
- Historial.

Fase 2: Productividad Legal

- Generacion de informes.
- Comparacion normativa.
- Exportacion DOCX/PDF.
- Alertas de inconsistencias.
- Relaciones entre documentos.

Fase 3: Gestion Normativa Avanzada

- Control de versiones.
- Vigencia documental.
- Comparacion entre versiones.

- Alertas por documentos derogados.
- Respuesta segun fecha de vigencia.

Fase 4: Produccion Institucional

- Auditoria avanzada.
- Roles granulares.
- Integracion con Google Drive o S3.
- Monitoreo.
- Backups.
- Panel de metricas.
- Hardening de seguridad.

16. Criterios de Aceptacion del MVP

- El administrador puede subir un PDF.
- El archivo se guarda en storage.
- El documento queda registrado en base de datos.
- El sistema extrae texto.
- El sistema clasifica el documento.
- El sistema genera metadata sugerida.
- El sistema genera resumen.
- El sistema fragmenta el texto.
- El sistema crea embeddings.
- El sistema indexa en Pinecone.
- El usuario puede hacer una pregunta.
- El sistema recupera fragmentos relevantes.
- La IA responde con fuentes.
- La respuesta muestra documentos, paginas o articulos.
- El historial queda guardado.
- Si no hay informacion suficiente, el sistema lo indica.

17. Recomendacion de Implementacion Inicial

Implementar primero estos cinco modulos:

1. Usuarios y seguridad
2. Biblioteca documental
3. Procesamiento IA
4. Busqueda semantica
5. Chat juridico con fuentes

Estos modulos forman el nucleo del sistema. Las demas funcionalidades deben construirse encima de este nucleo.

18. Principios de Mantenibilidad

- Cada modulo debe tener responsabilidades claras.
- El backend debe exponer APIs estables.
- Los servicios externos deben estar encapsulados.
- Los prompts deben ser tratados como artefactos versionados.
- Los resultados IA deben ser trazables.
- El usuario debe poder corregir metadata y datos extraidos.
- Las respuestas juridicas deben ser verificables.
- El sistema debe fallar de forma explicita y recuperable.